Projet Java EE — ClinicManager

Cahier des Charges & Spécifications Techniques Applicatives

1) Contexte & Vision

Contexte

Développer une application web Java EE permettant à une clinique de gérer :

- les patients ;
- les médecins ;
- les spécialités et départements ;
- les disponibilités ;
- les rendez-vous ;
- les notes médicales.

Vision

Construire une application structurée, maintenable et testable respectant une architecture multicouche :

```
JSP / JSTL
      ↓
Servlets
      ↓
Services
      ↓
Repositories
      ↓
JPA / Hibernate
      ↓
PostgreSQL
```

2) Objectifs pédagogiques

Le projet doit permettre de mettre en pratique :

- POO et relations entre objets ;
- interfaces et enums ;
- SOLID et séparation des responsabilités ;
- JPA / Hibernate ;
- JPQL ;
- Servlets / JSP / JSTL ;
- Sessions et Filters ;
- LocalDate, LocalTime, LocalDateTime ;
- Collections et Streams ;
- DTO / Mapper ;
- exceptions métier ;
- JUnit 5 / Mockito.

3) Configuration technique

Technologies

- Java 17+
- Jakarta EE
- Maven
- Servlet
- JSP / JSTL
- JPA / Hibernate
- PostgreSQL
- Tomcat
- JUnit 5
- Mockito
- Git

Structure générale

```
clinic-manager/
│
├── pom.xml
│
├── src/main/java/
│   └── com/clinicmanager/
│       ├── model/
│       ├── dto/
│       ├── mapper/
│       ├── repository/
│       ├── service/
│       ├── controller/
│       ├── filter/
│       ├── exception/
│       └── util/
│
├── src/main/resources/
│   └── META-INF/
│       └── persistence.xml
│
├── src/main/webapp/
│   ├── WEB-INF/
│   ├── css/
│   ├── auth/
│   ├── patient/
│   ├── doctor/
│   └── admin/
│
└── src/test/java/
```

4) Phase 1 — Authentification & Utilisateurs

Fonctionnalités

Inscription

Le patient fournit :

- nom ;
- prénom ;
- email ;
- téléphone ;
- mot de passe.

Connexion

Connexion avec :

- Email + mot de passe
- Utilisation d'une HttpSession.

Profil

L'utilisateur peut :

- consulter son profil ;
- modifier ses informations ;
- changer son mot de passe.

Règles principales

- email unique ;
- mot de passe ≥ 6 caractères ;
- compte désactivé → connexion interdite.

5) Phase 2 — Gestion des patients

Un patient possède :

- CIN
- Nom
- Prénom
- Date de naissance
- Genre
- Adresse
- Téléphone
- Groupe sanguin

Le patient peut :

- consulter son profil ;
- modifier ses informations ;
- consulter ses rendez-vous ;
- consulter son historique médical.

L'administrateur peut gérer les patients.

6) Phase 3 — Gestion des médecins

Un médecin possède :

- Matricule
- Nom

- Prénom
- Titre
- Email
- Téléphone
- Spécialité
- Département
- Le matricule doit être unique.

Le médecin peut :

- consulter son agenda ;
- gérer ses disponibilités ;
- gérer ses absences ;
- consulter ses rendez-vous ;
- créer des notes médicales.

7) Phase 4 — Spécialités & Départements

L'administrateur peut gérer :

Départements

- Médecine
- Chirurgie
- Pédiatrie
- Urgences
- ...

Spécialités

- Cardiologie
- Dermatologie
- Pédiatrie
- Neurologie
- ...

Relation :

> Department
> 1
> |
> └──────── *
> Specialty

8) Phase 5 — Disponibilités & Créneaux

Le médecin définit ses horaires.

Exemple :
• Lundi
08:30 → 12:00
14:00 → 17:30

Une disponibilité contient :

- Médecin
- Jour

- Heure début
- Heure fin
- Statut
- Période de validité

Configuration

- Durée rendez-vous : 30 min
- Buffer : 5 min
- Lead time : 2 h
- Pause : 12:00 → 13:00
- Dimanche : fermé

Le système génère automatiquement les créneaux disponibles.

Il doit exclure :

- les congés ;
- les absences ;
- les rendez-vous existants ;
- la pause déjeuner ;
- les créneaux trop proches.

9) Phase 6 — Gestion des rendez-vous

Le patient sélectionne :

- Spécialité
- Médecin
- Date
- Créneau
- Type
- Motif

Types :

- CONSULTATION
- FOLLOW_UP
- URGENT

Statuts :

- PLANNED
- DONE
- CANCELED

Fonctionnalités

- créer ;
- consulter ;
- modifier ;
- annuler ;
- replanifier.

Règles principales

Un rendez-vous :

- doit être dans une disponibilité ;

- ne doit pas chevaucher un autre rendez-vous ;
- doit respecter le lead time de 2 heures ;
- ne peut être annulé par le patient que 12 heures avant ;
- reste dans l'historique après annulation.

10) Phase 7 — Notes médicales

Après une consultation, le médecin peut créer une note.

> Appointment
>
> ↓
>
> DONE
>
> ↓
>
> MedicalNote

Une note contient :

- Médecin
- Rendez-vous
- Diagnostic
- Contenu
- Date

Une note validée devient :

- READ ONLY (Elle ne peut plus être modifiée).

11) Phase 8 — Architecture & Persistance

Entités principales

- User
- Patient
- Doctor
- Department
- Specialty
- Availability
- Appointment
- MedicalNote

Repositories

Créer :

- UserRepository
- PatientRepository
- DoctorRepository
- DepartmentRepository
- SpecialtyRepository
- AvailabilityRepository
- AppointmentRepository
- MedicalNoteRepository

> **Repository = accès aux données**

12) Phase 9 — Services & Règles métier

Créer :

- AuthService
- PatientService
- DoctorService
- DepartmentService
- SpecialtyService
- AvailabilityService
- TimeSlotService
- AppointmentService
- MedicalNoteService

> **Service = logique métier**

Exemple :

> AppointmentService
>
> ↓
>
> Vérifier patient
>
> ↓
>
> Vérifier médecin
>
> ↓
>
> Vérifier disponibilité
>
> ↓
>
> Vérifier conflit
>
> ↓
>
> Vérifier délai
>
> ↓
>
> Créer rendez-vous

13) Phase 10 — Interface Web

Créer les principales interfaces :

Patient

- Dashboard
- Mon profil
- Prendre rendez-vous
- Mes rendez-vous
- Historique

Médecin

- Dashboard
- Mon agenda
- Mes disponibilités
- Mes rendez-vous
- Notes médicales

Admin

- Dashboard
- Patients
- Médecins
- Spécialités
- Départements
- Utilisateurs

STAFF

- Planning
- Rendez-vous
- Replanification
- Liste d'attente

14) Phase 11 — Sécurité & Autorisation

Utiliser :

- HttpSession
- Filters
- Roles

Exemple :

```
/admin/*     → ADMIN
/doctor/*    → DOCTOR
/patient/*   → PATIENT
/staff/*     → STAFF
```

- Un utilisateur ne doit accéder qu'aux fonctionnalités autorisées par son rôle.

15) Phase 12 — DTO, Mapper & Exceptions

Les JSP ne doivent pas manipuler directement les entités JPA.

Architecture :

```
Entity
  ↓
Mapper
  ↓
DTO
  ↓
Servlet
  ↓
JSP
```

Exceptions métier à créer :

- UserNotFoundException
- DoctorNotFoundException
- AppointmentNotFoundException
- AppointmentConflictException

- UnavailableSlotException
- DuplicateEmailException
- DuplicateMatriculeException
- MedicalNoteLockedException
- UnauthorizedActionException

16) Phase 13 — Tests

Utiliser :

- JUnit 5
- Mockito

Tester principalement :

AppointmentService

- rendez-vous valide ;
- conflit ;
- médecin indisponible ;
- délai insuffisant ;
- annulation interdite ;
- modification.

TimeSlotService

- génération des créneaux ;
- exclusion des rendez-vous ;
- exclusion des congés ;
- exclusion de la pause.

MedicalNoteService

- note sur rendez-vous DONE ;
- note interdite sur rendez-vous PLANNED ;
- modification interdite après validation.

17) Phase 14 — Livrables

Les apprenants doivent fournir :

- ✓ Projet Maven
- ✓ Code source
- ✓ Base PostgreSQL
- ✓ persistence.xml
- ✓ Script SQL
- ✓ Tests JUnit / Mockito
- ✓ README
- ✓ Documentation
- ✓ WAR déployable sur Tomcat
- ✓ Dépôt Git

18) Scénarios principaux

Scénario 1 — Prise de rendez-vous

> Patient

↓
Login
↓
Choisir spécialité
↓
Choisir médecin
↓
Choisir date
↓
Afficher créneaux
↓
Choisir créneau
↓
Vérification
↓
Rendez-vous PLANNED

Scénario 2 — Consultation

Médecin
↓
Agenda
↓
Rendez-vous
↓
Consultation terminée
↓
DONE
↓
Créer note médicale
↓
VALIDATED

Scénario 3 — Annulation

Patient
↓
Mes rendez-vous
↓
Choisir rendez-vous
↓
Vérifier délai ≥ 12h
↓
CANCELED

19) Bonus

Pour les apprenants avancés :

- **Bonus 1 :** Liste d'attente automatique.
- **Bonus 2 :** Notification email de confirmation.
- **Bonus 3 :** Rappel 24h avant le rendez-vous.
- **Bonus 4 :** Dashboard avec statistiques.